

# Mikrocomputertechnik 1

## Einheit 5: Der Single-Cycle-Datenpfad

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 79

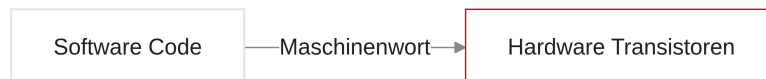
### Der Single-Cycle-Datenpfad

Folie

2 / 79

#### Von der Instruktion zur Hardware

- Herzlich willkommen zur fünften Vorlesung in Mikrocomputertechnik 1
- Bisher: Fokus auf die Software-Sicht (Assembler)
- Heute: Brückenschlag zurück zur Hardware
- Die Blackbox des RISC-V-Prozessors öffnen
- Ziel: verstehen, wie eine Textzeile physikalisch Strom durch Leitungen leitet

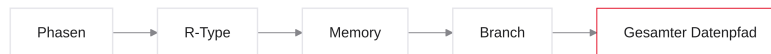


#### Notizen

Herzlich willkommen zu Einheit 5. Assembler, Schleifen und Stack beherrschen Sie. Heute: Wie leitet der Chip aus 32 Bits ab, welche Leitungen unter Strom stehen? Logikgatter aus Einheit 1 verschmelzen mit dem Assembler aus Einheit 4.

## Agenda für den heutigen Tag

1. Die 5 Phasen — Lebenszyklus eines Maschinenbefehls
2. Fetch und R-Type — Hardware für Berechnungen
3. Load und Store — Arbeitsspeicher physikalisch einbinden
4. Der Branch-Pfad — dem PC das Springen beibringen
5. Das Gesamtbild — Zusammenführung über Multiplexer
6. Ripes — visuelles Tracing auf den Logikgattern



## Notizen

Bottom-up: erst die fünf Phasen, dann Bauteil für Bauteil von ADD über Speicher bis Branches. Am Ende setzen wir den Datenpfad in Ripes unter Strom.

## Rückblick: Was wir bisher blind genutzt haben

- Maschinenbefehl immer exakt 32 Bit breit
- Instruction Formats: R, I, S, B
- Register lesen und überschreiben (`t0`, `s0`, ...)
- ALU kann addieren und vergleichen
- PC als logischer Zeiger im Code

```
# Wir wissen, WAS dieser Befehl tut:  
add t0, t1, t2
```

```
# Aber WIE wird aus 0x007302B3 die ALU aktiviert?
```

## Notizen

Bisher Magie: Wer öffnet das Register File? Wer konfiguriert die ALU? Die CPU war Blackbox — heute öffnen wir sie.

## Die Blackbox öffnen

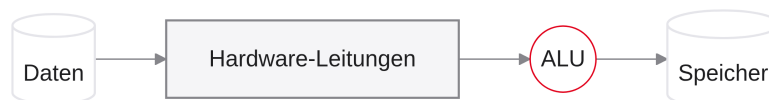
- Ein realer Prozessor: Milliarden Transistoren
- Mikroskopische „Autobahnen“ — für Menschen nicht greifbar
- Abstraktion: Blockdiagramme statt einzelner Elektronen
- Fokus: Datenflüsse und Vernetzung der Komponenten

## Notizen

Ingenieure abstrahieren zu Blöcken: Register File, ALU, Multiplexer — verbunden durch 32-Bit-Leitungsbündel.

## Was ist ein Datenpfad (Datapath)?

- Physikalische „Rohrleitung“ des Prozessors
- Alles, was Daten verarbeitet oder transportiert: ALU, Register File, Speicher, Multiplexer
- Nicht dazu: die Control Unit (Steuerwerk) — sie dirigiert nur
- Der Datenpfad begrenzt die Maximalleistung des Chips



## Notizen

Datenpfad = Nutzdaten-Fluss. Steuerwerk = Hebel an den Ventilen. Heute exklusiv das Rohrleitungssystem.

## Befehlsumfang dieses Lehrmodells

- Implementiert: `add`, `sub`, `and`, `or`, `lw`, `sw`, `beq`
- Kein vollständiger RV32I-Prozessor
- `addi`, `bne` und `jal` erst nach Ergänzung von ImmGen, Vergleich oder Link-Write-Back
- ImmGen liefert für `beq` den Byteoffset einschließlich implizitem Bit 0

## Notizen

Annahme unseres Lehrmodells. Ripes zeigt ein reicheres Modell; die Folien behaupten nur das genannte

Subset. Die Zuordnung der Signalnamen steht in der Mapping-Tabelle weiter unten, nicht als Identität mit Ripes.

## Folie

8 / 79

### Der Single-Cycle-Prozessor

- Viele Steuerungsarten: Pipelining, Multi-Cycle, ...
- Heute: didaktisch zentrales Modell — Single-Cycle
- Ein Maschinenbefehl in exakt einem Taktzyklus
- Bei jeder steigenden Taktflanke ist ein Befehl fertig
- Leicht verständlich, modern oft ineffizient (Vertiefung später)

## Notizen

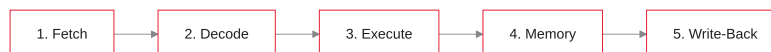
Ein Takt: Fetch bis Write-Back. Moderne Chips arbeiten anders — das Modell bleibt der Schlüssel zum Verständnis.

## Folie

9 / 79

### Die 5 Phasen

- Logisch fünf Stationen, auch wenn alles in einem Takt geschieht
- Universelles Schema vieler Architekturen
- Datenfluss strikt von links nach rechts (Write-Back als Rückführung)
- Abkürzungen: IF, ID, EX, MEM, WB



## Notizen

Heiliger Gral der Rechnerarchitektur: Instruction Fetch, Decode, Execute, Memory, Write-Back — von Mikrocontroller bis Server.

### Phase 1: Instruction Fetch (IF)

- 32-Bit-Maschinencode aus dem Speicher holen
- PC liefert die physikalische Adresse
- Instruction Memory liefert die Instruktion
- Parallel: Vorbereitung  $PC + 4$  für den nächsten Befehl



#### Notizen

Noch „dumm“: Adresse lesen, Wort liefern. Ein kleiner Addierer hält bereits  $PC + 4$  bereit.

### Phase 2: Instruction Decode (ID)

- 32-Bit-Wort entschlüsseln, Operanden bereitstellen
- Felder: opcode,  $rs1$ ,  $rs2$ , ...
- $rs1/rs2$  an das Register File — Werte auf die Datenleitungen
- Control Unit erzeugt Steuersignale aus dem Opcode

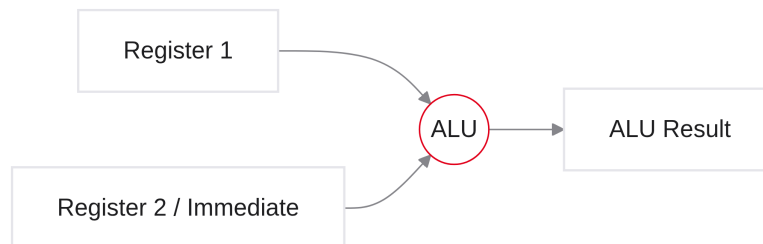


#### Notizen

Opcode analysieren, Registeradressen herausgreifen, Werte auf RD1/RD2 legen — kombinatorisch (reine Gatterlaufzeiten, ohne Taktzustand).

### Phase 3: Execute (EX)

- Mathematische oder logische Arbeit: die ALU
- R-Type: zwei Registerwerte verknüpfen
- Load/Store: Zieladresse = Base + Offset
- Branch: Vergleich (z. B. Differenz auf Null)

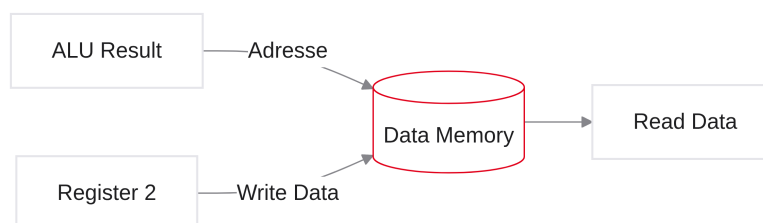


#### Notizen

Die ALU rechnet nicht nur bei `add`: bei `lw/sw` berechnet sie die RAM-Adresse (Base + Offset).

### Phase 4: Memory (MEM)

- Kommunikation mit dem Datenspeicher
- Relevant für `lw`, `sw` und Verwandte
- Andere Befehle: Bypass am RAM vorbei
- `lw`: lesen; `sw`: schreiben an ALU-Adresse



#### Notizen

Bei `add` bleibt der Datenspeicher abgeriegelt; das ALU-Ergebnis fließt parallel weiter zum Write-Back.

### Phase 5: Write-Back (WB)

- Fertigen Wert in Zielregister `rd` ablegen
- Rückführung zum Register File (visuell „nach links“)
- `add`: ALU-Ergebnis; `lw`: RAM-Daten
- `beq/sw`: kein Register-Schreiben — Phase inaktiv



#### Notizen

Mit dem Write-Back ist der Befehl vollständig abgearbeitet — bereit für den nächsten Takt.

### Fetch und R-Type-Datenpfad

- Aufbau des Hardware-Schaltplans beginnen
- Fetch: Instruktion holen
- PC und PC+4-Addierer
- Verschaltung für `add/sub`
- Timing innerhalb eines Taktes



#### Notizen

Vom PC links nach rechts zum reinen Rechenpfad — R-Type ohne Datenspeicher.

## Der Startpunkt: Der PC

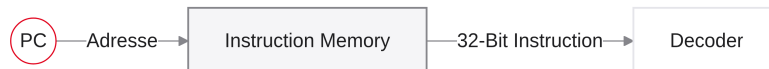
- Program Counter: spezielles 32-Bit-Register
- Speichert den Ausführungszustand (State)
- D-Flip-Flops: Update nur bei positiver Taktflanke
- Während des Taktes: stabile Adresse am Ausgang

### Notizen

Anker des Datenpfads. Flankengetriggert: neue Adresse einfrieren, bis zum nächsten Tick halten.

## Instruction Memory (Befehlsspeicher)

- Erstes Bauteil hinter dem PC
- PC-Wert am Adress-Eingang
- Single-Cycle-Modell: kombinatorische Ausgabe der Instruktion
- Während der Ausführung: nur Lesen

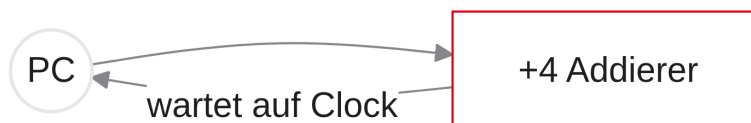


### Notizen

Adresse anlegen — nach kurzer Verzögerung liegt das 32-Bit-Wort am Ausgang. Fetch erledigt.

## PC-Inkrementer ( $PC + 4$ )

- Separater Hardware-Addierer (nicht die Haupt-ALU)
- Fest verdrahtet: immer  $+4$  zur PC-Adresse
- Ergebnis zurück an den PC-Eingang — wartet auf die nächste Flanke



## Notizen

Kabel splitten: ein Strang zum Instruction Memory, einer zum +4-Addierer. Flip-Flops übernehmen erst beim Tick.

## Folie

19/79

### R-Type-Refresher

- Instruktion liegt auf den Leitungen
- Ziel: `add rd, rs1, rs2` physikalisch
- Bits 19:15 = `rs1`, 24:20 = `rs2`, 11:7 = `rd`

| Bits  | Feld           |
|-------|----------------|
| 31:25 | funct7         |
| 24:20 | rs2 (lesen)    |
| 19:15 | rs1 (lesen)    |
| 14:12 | funct3         |
| 11:7  | rd (schreiben) |
| 6:0   | opcode         |

## Notizen

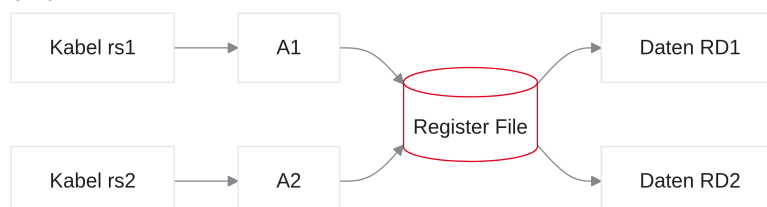
32-Bit-Kabel aufspließen: drei 5-Bit-Bündel für `rs1`, `rs2`, `rd` zum Register File.

## Folie

20/79

### Register lesen (ID)

- Zwei Lese-Ports A1 und A2
- `rs1/rs2` direkt angeschlossen
- Kombinatorisch: Inhalte auf RD1 und RD2
- Operanden physikalisch isoliert

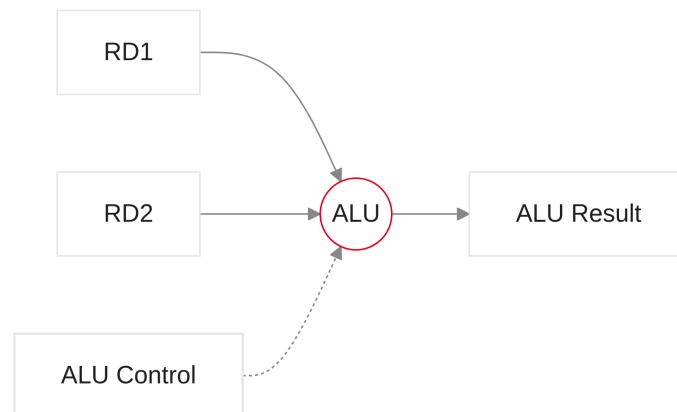


## Notizen

Asynchron in der ID-Phase: Spannung an den Ports — Werte auf den Datenausgängen.

## Die ALU (EX)

- RD1 und RD2 in die ALU-Eingänge
- Sofortige Verknüpfung (kombinatorisch)
- Operation via ALU-Control (aus opcode/funct3)
- Ergebnis am ALU-Ausgang



### Notizen

Kein Zwischenspeicher in der ALU: Gatterlaufzeiten, dann steht das Resultat. Execute-Phase erreicht.

## Write-Back (WB)

- Ergebnis zurück zur Werkbank
- Leitung vom ALU-Ausgang zum WD-Port (Write Data)
- Visuell der „rückwärts“-Pfad auf dem Plan
- Bits warten am Register File

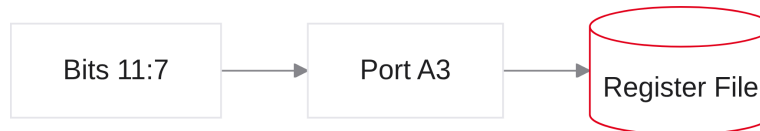


### Notizen

Bei R-Type wird MEM umgangen. Daten stehen an der Tür — noch nicht geschrieben.

### Zieladresse ( $rd$ )

- Schreibfach: Bits 11:7 ( $rd$ ) an Port A3
- Hardware für R-Type damit vollständig verdrahtet



### Notizen

A3 sagt: „Schreibe die anliegenden Daten in Fach Nummer  $rd$ .“

### Das RegWrite-Signal

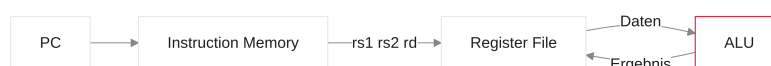
- Schutzventil: Steuerpin RegWrite (1 Bit)
- RegWrite = 1: bei nächster Flanke in  $rd$  speichern
- RegWrite = 0: Signale prallen ab (wichtig für  $sw/beq$ )

### Notizen

Ohne RegWrite würde jedes Tick Register überschreiben. Nur bei erlaubtem Write-Back auf 1.

### Der fertige R-Type-Pfad

- Strom: PC → Instruction Memory → Register lesen → ALU → Write-Back
- Kein Zugriff auf den Data Memory

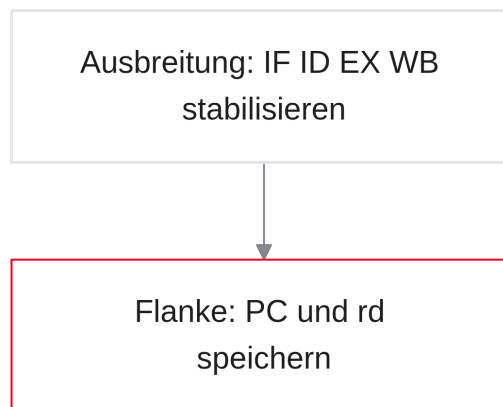


### Notizen

Synchronisierter Datenfluss ohne RAM. Als Nächstes: Timing — wann wird wirklich gespeichert?

## Timing im Single-Cycle

- Gatter arbeiten ununterbrochen; Signale breiten sich aus
- Erst wenn EX/WB stabil: Ergebnis am Register-Eingang
- Steigende Flanke am Taktende:
  - Register File speichert (wenn  $\text{RegWrite} = 1$ )
  - PC übernimmt  $PC + 4$
- Sofort Start des nächsten Befehls



## Notizen

Während der Periode: Kombinatorik. Flanke: Zustände einfrieren. (Gantt-Darstellung bewusst als Phasen-Kette — handout-tauglich.)

## Warum überschreiben wir nichts vorzeitig?

- Fall:  $rd = rs1$ , z. B. `addi t0, t0, 1`
- Alter Wert wird gelesen, durch ALU, steht am Schreib-Port
- Flip-Flops ändern den Ausgang erst mit der Flanke
- Deshalb ist  $a = a + b$  gefahrlos

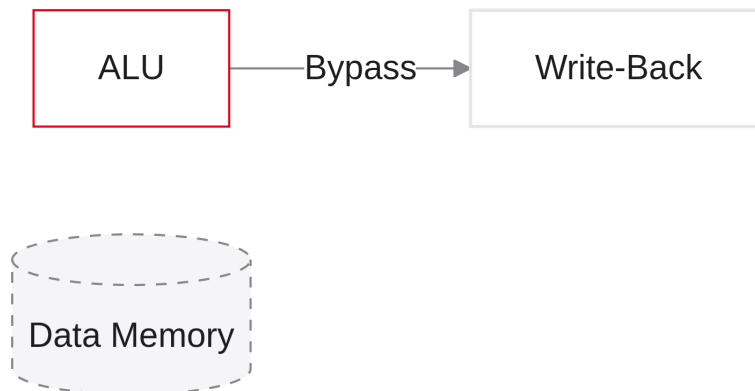
```
t0 = t0 + t1;  
// Lesen und Schreiben im selben Takt - Flankensteuerung rettet uns
```

## Notizen

Klassische Klausurfrage: Lesen und Schreiben desselben Registers im Takt. Flankensteuerung hält den alten Wert stabil bis zum Tick.

### Was passiert mit Phase 4 (MEM)?

- Im reinen R-Type-Pfad fehlt der Data Memory
- MEM existiert logisch, physikalisch Bypass
- ALU-Ergebnis direkt zum Write-Back
- Nächster Schritt: RAM für Load/Store einhängen, ohne R-Type zu zerstören

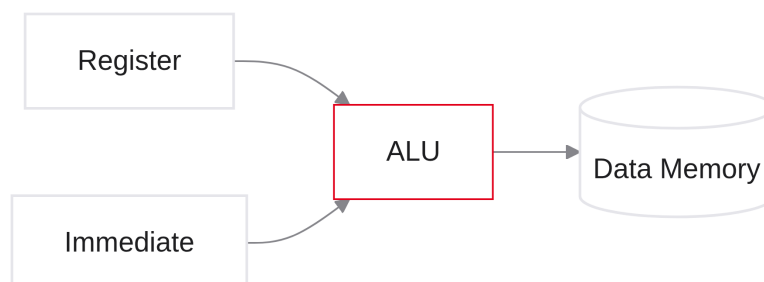


### Notizen

Ungenutzte Blöcke bleiben abgemeldet. Load/Store brauchen denselben Pfad — elegant erweitert, nicht neu erfunden.

### Der Load/Store-Datenpfad

- Konstanten (Immediates) in die ALU
- Data Memory einbinden
- Erster Ressourcenkonflikt — Multiplexer
- Datenwege für `lw` und `sw`
- Steuersignale `MemRead` / `MemWrite`



## Notizen

Neue Leitungen erzeugen Kollisionen an ALU-Eingängen und Write-Back. Multiplexer als elektrische Weichen — nächster Bauabschnitt.

## Folie

30 / 79

### Das Problem mit I-Types

- R-Type: beide Operanden aus dem Register File ( $rs1, rs2$ )
- I-Type ( $lw, addi$ ): kein  $rs2$  — stattdessen 12-Bit-Immediate
- Die ALU hat 32-Bit-Eingänge
- Die Konstante muss aus dem Befehlswort in die ALU gelangen

| Bits  | Feld                                                    |
|-------|---------------------------------------------------------|
| 31:20 | $imm[11:0] \rightarrow$ diese 12 Bits müssen in die ALU |
| 19:15 | $rs1$                                                   |
| 14:12 | $funct3$                                                |
| 11:7  | $rd$                                                    |
| 6:0   | $opcode$                                                |

## Notizen

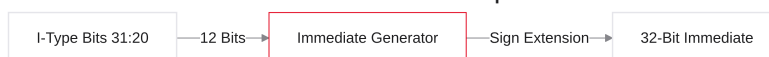
Für  $lw$  braucht die ALU Base + Offset. 12 Drähte an einen 32-Bit-Eingang zu löten reicht nicht — 20 Leitungen fehlen. Sign-Extension in Hardware ist nötig.

## Folie

31 / 79

### Immediate Generator (ImmGen)

- Vereinfacht für I-Type: Bits 31:20 als 12-Bit-Immediate
- S-/B-Formate nutzen andere Bitfelder — ImmGen kennt mehrere Layouts
- 12-Bit-Eingang, 32-Bit-Ausgang
- Aufgabe: Vorzeichenerweiterung (Sign-Extension) in Hardware
- Beim I-Type: Bit 31 wird auf die oberen 20 Bits kopiert



## Notizen

Einstieg am I-Type ( $addi, lw$ ): ImmGen fächert das Vorzeichenbit auf. Bei Branches/Stores greifen andere Immediate-Layouts — dasselbe Bauteil, andere Verdrahtung der Instruktionsbits.

### Der erste Konflikt (Kollision)

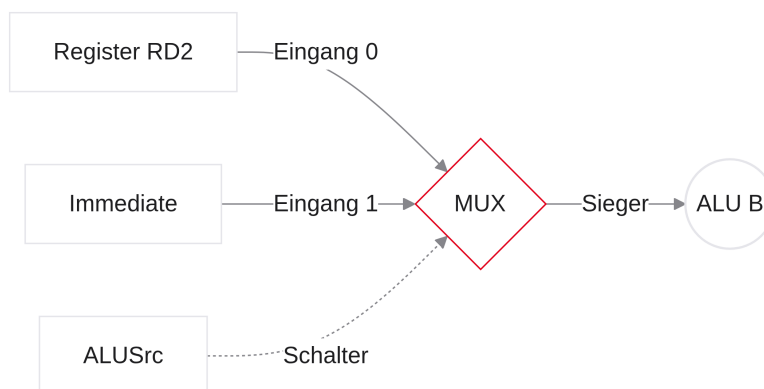
- Zwei 32-Bit-Signale wollen denselben ALU-B-Eingang
- `add`: Signal aus RD2 (Register File)
- `lw/addi`: Signal aus ImmGen
- Beide Kabel auf einen Kontakt = Kurzschluss

### Notizen

Klassischer Ressourcenkonflikt. Lösung: elektronische Weiche — Multiplexer.

### MUX 1: ALU-Source (**ALUSrc**)

- Multiplexer: mehrere Eingänge, ein Ausgang
- Steuerpin `ALUSrc`
- `ALUSrc = 0`: RD2 zur ALU
- `ALUSrc = 1`: Immediate zur ALU



### Notizen

MUX wie eine Bahnweiche: Control Unit stellt `ALUSrc`, Kurzschluss vermieden.

## Load Word (lw) — die Adressierung

- Mit MUX kann die ALU Konstanten verarbeiten
- Bei lw: ALU berechnet nur die RAM-Adresse (Base + Offset)
- Ergebnis = Hausnummer, kein „normales“ Rechenergebnis
- Diese Adresse geht an den Datenspeicher

```
lw t0, 12(s1)
# 1. ALUSrc = Immediate (12)
# 2. ALU: s1 + 12
# 3. Adresse am ALU-Ausgang
```

### Notizen

Base am A-Eingang, Offset über ImmGen/MUX am B-Eingang — Adresse für den Data Memory.

## Data Memory (Datenspeicher)

- Block in der MEM-Phase: Data Memory (RAM)
- Address: Ausgang der ALU
- Write Data: für Store
- Read Data: gelesene Daten
- Im Modell: kombinatorisches Lesen (wie Instruction Memory)



### Notizen

Didaktisches Modell: getrennte Instruction- und Data-Memory-Blöcke. Adresse anlegen — Daten auf Read Data.

### Daten aus dem RAM lesen

- Read Data = Wert des `lw`
- Write-Back: Leitung zurück zum WD-Port des Register Files
- Ziel: bei der Flanke in `rd` speichern
- Kollision: dort liegt bereits das ALU-Rückführkabel



### Notizen

Zwei Kandidaten am WD-Port: ALU-Ergebnis (R-Type) vs. RAM-Daten (`lw`).

### Der zweite Konflikt (`MemoReg`)

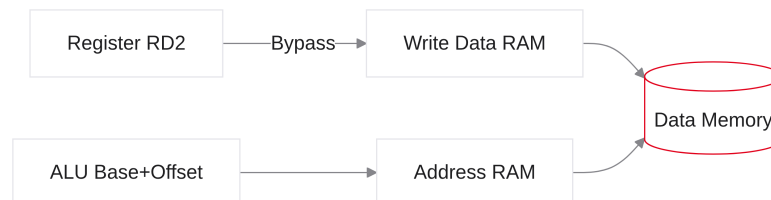
- WD empfängt nur ein 32-Bit-Signal
- R-Type will ALU-Ergebnis; `lw` will RAM-Daten
- Lösung: zweiter MUX vor dem Register File
- Steuersignal: `MemoReg`

### Notizen

`MemoReg` = 1: RAM durchlassen; = 0: ALU durchlassen.

## Store Word (sw) — der Pfad

- sw: rs2 liefert die zu speichernden Daten
- Wert liegt schon auf RD2 (ID-Phase)
- Abzweig von RD2 zum Write-Data-Port des RAM (Bypass um die ALU)
- ALU berechnet parallel Base + Offset als Adresse



### Notizen

Adresse wie bei lw; Datenpfad von RD2 direkt in den RAM-Schreibeingang.

## Die Signale MemWrite und MemRead

- Speicher braucht Schutzpins wie das Register File
- MemWrite = 1: bei Flanke schreiben
- MemRead = 1: Daten auf Read Data ausgeben
- Bei add: beide 0 — RAM inaktiv

### Notizen

RD2 wackelt oft am Speicher. Nur bei echtem Store öffnet MemWrite den Schreibpfad.

## Store und das Register File

- sw schreibt in den RAM, nicht in Register
- S-Type hat kein rd; dieselben Bits sind Immediate und rs2
- RegWrite muss 0 sein, sonst wird das aus den Bits decodierte Register beschrieben

Ohne RegWrite=0:  
MUX MemtoReg leitet immer etwas weiter -  
Store wuerde Register mit Muell beschreiben.

## Notizen

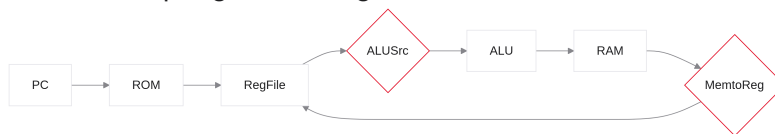
Zusammenspiel der Steuersignale: Store = RAM schreiben, Register File abgeriegtelt.

## Folie

41 / 79

### Der Load/Store-Pfad erweitert

- Rechnen (R-Type) und Speicherzugriff (I-/S-Type)
- Multiplexer regeln den Verkehr
- Es fehlt: dem PC das Springen beibringen



## Notizen

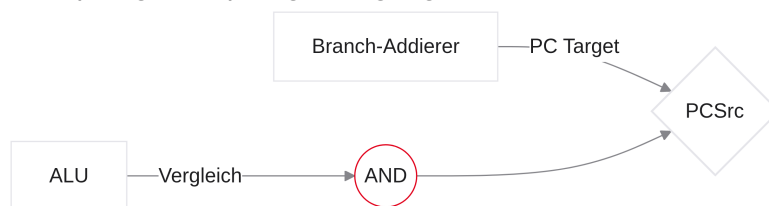
Lineare Programme sind machbar. Nächste Stufe: Kontrollfluss — Branch-Datenpfad.

## Folie

42 / 79

### Der Branch-Datenpfad

- Hardware für bedingte Entscheidungen
- Problemstellung `beq`
- ALU als Komparator (Zero-Ausgang)
- Zweiter Addierer für das Branch-Target
- Logische Verknüpfung der Sprungbedingung



## Notizen

Register vergleichen und PC dynamisch setzen — wenn die Bedingung wahr ist.

### Das Branch-Problem (beq)

- `beq rs1, rs2, Label`: zwei Aufgaben in einem Takt
- Bedingung: sind `rs1` und `rs2` gleich?
- Navigation: neues PC-Ziel = aktueller PC + Offset
- Beides braucht Addition/Subtraktion — nur eine Haupt-ALU

```
# Hardware-Sicht:  
# 1. rs1 == rs2 ?  
# 2. PC_neu = PC + Offset  
# wenn (1): PC = PC_neu
```

#### Notizen

Ressourcenkonflikt: Vergleich und Target-Berechnung gleichzeitig — zweiter Addierer nötig.

### Vergleich in Hardware (Subtraktion)

- Trick: Subtraktion statt direktem Gleichheitstest
- ALU: Ergebnis =  $rs1 - rs2$
- Identisch  $\Rightarrow$  Ergebnis = 0
- Dann war `beq` erfolgreich

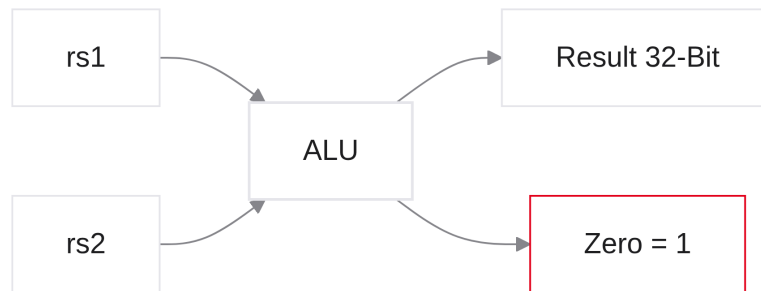
```
Wenn A == B, dann A - B == 0
```

#### Notizen

Ungleich null  $\Rightarrow$  ungleich. Hardware prüft, ob das ALU-Ergebnis nur Nullen enthält.

## Die ALU und der Zero-Ausgang

- Dedizierter 1-Bit-Ausgang: Zero (oft „Zero-Flag“ genannt)
- NOR über alle 32 Ergebnisbits
- Alle Bits 0  $\Rightarrow$  Zero = 1
- Sonst Zero = 0
- Das Datenwort der ALU wird bei Branches typisch verworfen



### Notizen

Hinweis Terminologie: In RISC-V-Datenpfaden ist Zero ein ALU-Ausgangspin, kein separates Statusregister wie bei x86. Ein Draht = Wahrheitsbit für den Branch.

## Zieladresse berechnen (Navigation)

- Haupt-ALU ist mit der Subtraktion belegt
- Offset steckt im B-Type als zerrissenes Immediate
- ImmGen setzt zusammen und sign-extended auf 32 Bit
- Negative Offsets: Sprünge „nach oben“ (Schleifen)

| Bits  | Feld                  |
|-------|-----------------------|
| 31:25 | imm[12] und imm[10:5] |
| 24:20 | rs2                   |
| 19:15 | rs1                   |
| 14:12 | funct3                |
| 11:7  | imm[4:1] und imm[11]  |
| 6:0   | opcode                |

### Notizen

ImmGen kennt auch das B-Format. Zweierkomplement erlaubt rückwärts gerichtete Sprünge.

## Offset-Aufbereitung im Lehrmodell

- ImmGen liefert den vorzeichenerweiterten **Byteoffset**
- Bit 0 ist implizit 0 und bereits enthalten
- Danach kein zweites Shift-left-1
- Der Branch-Addierer rechnet alten PC plus diesen Byteoffset

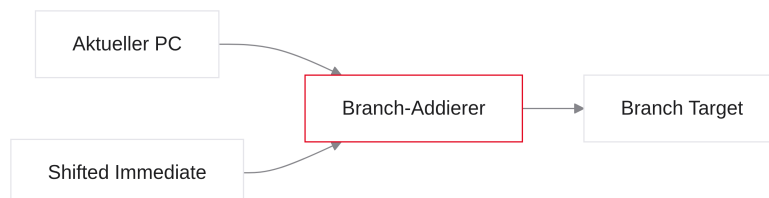
```
Byteoffset = signextend(B-Immediate mit implizitem Bit 0)
Target     = oldPC + Byteoffset
```

### Notizen

Vereinfachte Darstellung für RV32I: Wortausrichtung und B-Immediate-Skalierung. Details im Lehrbuchdiagramm (Patterson & Hennessy) und in Ripes nachvollziehbar.

## Der Branch-Addierer

- Zweiter, dedizierter Addierer (Haupt-ALU blockiert)
- Eingang 1: aktueller PC
- Eingang 2: aufbereitetes Offset
- Ausgang: Branch Target



### Notizen

Silizium für Parallelität: Vergleich und Target-Berechnung im selben Takt.

### Der dritte Konflikt (PC-Update)

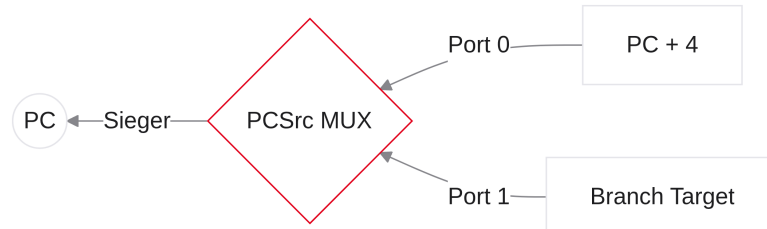
- Am Taktende braucht der PC eine neue Adresse
- Kandidat 1:  $PC + 4$
- Kandidat 2: Branch Target
- Zwei Kabel am PC-Eingang — wieder MUX nötig

#### Notizen

Dritter Weichensteller: direkt vor dem PC-Register.

### MUX 3: PC-Source (PCSrc)

- MUX vor dem PC-Eingang
- Port 0: linear ( $PC + 4$ )
- Port 1: Branch Target
- Steuersignal: PCSrc

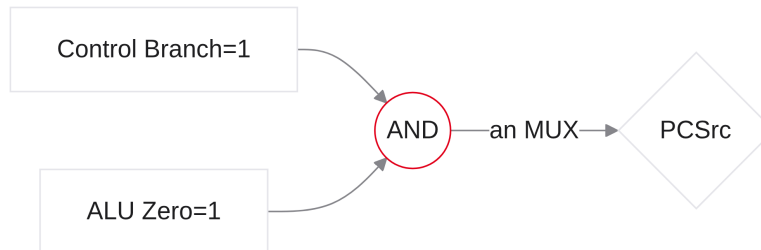


#### Notizen

Wer stellt PCSrc? Nur bei echtem, erfolgreichem Branch umschalten.

## Wer steuert PCSrc? (Die Logik)

- Zwei Bedingungen gleichzeitig:
  - Befehl ist Branch (Control: Branch = 1)
  - Vergleich erfolgreich (ALU: Zero = 1)
- Verknüpfung mit AND-Gatter
- $PCSrc = Branch \wedge Zero$

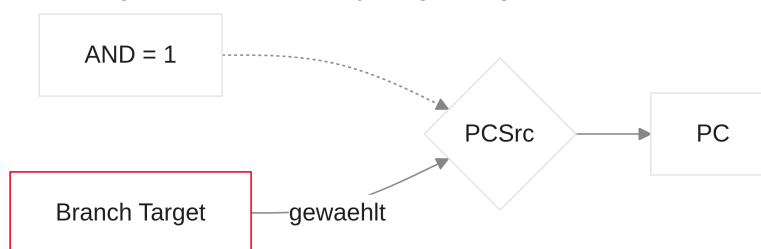


### Notizen

Zufällige ALU-Null bei `sub` darf nicht springen — nur Branch-Opcode plus Zero.

## Trace: Branch Taken

- Bedingung wahr: `beq`, Register gleich
- $AND = 1$ , MUX auf Port 1
- Nächste Flanke: Target in den PC — Sprung erfolgt

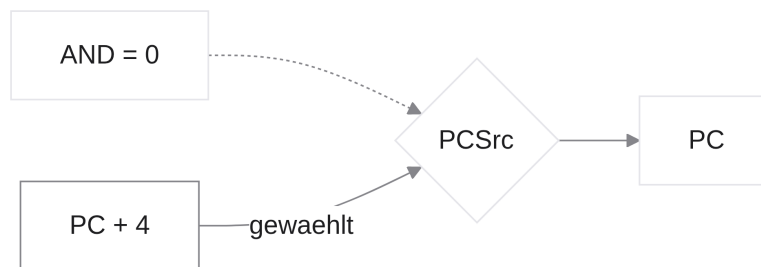


### Notizen

Zero leuchtet, Branch leuchtet, AND feuert, Weiche klappt — PC übernimmt das Label.

### Trace: Branch Not Taken (Fall-Through)

- Bedingung falsch: Register ungleich
- $\text{Zero} = 0 \rightarrow \text{AND} = 0$
- MUX bleibt auf Port 0:  $PC + 4$
- Target-Adresse verpufft ungenutzt



### Notizen

Fall-Through: lineare Fortsetzung. Eleganz reiner Kombinatorik plus einer Flanke.

### Unbedingte Sprünge (jal) — Ausblick

- `jal` braucht etwas mehr Verdrahtung als `beq`
- Bedingungslos: `PCSrc` gezielt auf Target
- Rückflugticket:  $PC + 4$  muss nach `rd` (meist `ra`)
- Extra-MUX/Kabel zum Write-Back — im Blockdiagramm vereinfacht
- In Ripes im Labor sichtbar

```

jal ra, Label, beide Werte aus dem alten PC:
  nextPC = oldPC + Byteoffset
  ra     = oldPC + 4
  
```

`jal` liegt außerhalb des Basismodells `add/sub/and/or/lw/sw/beq`, bis der Write-Back-Pfad für den Linkwert ergänzt ist. Die sequenzielle Schreibweise „erst `PC = Target`, dann `ra = PC+4`“ wäre falsch, weil der Linkwert den alten `PC` braucht.

### Notizen

Vereinfachung: Wir blenden die `jal`-Write-Back-Kabel hier aus; Einheit 4 und Ripes zeigen den Link-Pfad.

## Die Anatomie ist fast komplett

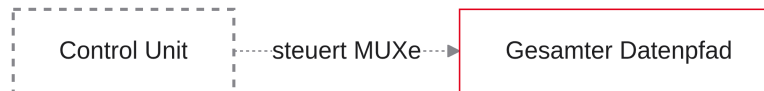
- Speicher/Register: PC, Instruction Memory, Data Memory, Register File
- Rechenwerke: Haupt-ALU, Branch-Addierer, +4-Addierer
- Weichen: drei Multiplexer (ALUSrc, MemtoReg, PCSrc)
- Branch-Logik: Zero und AND

### Notizen

Als Nächstes: Teilpfade zum Master-Diagramm und die Rolle der Control Unit.

## Der komplette Datenpfad

- Alle Teilpfade zu einem System
- Klassisches Architektur-Diagramm (Patterson & Hennessy)
- Control Unit als Dirigent der Steuersignale
- Schwäche des Single-Cycle: Timing / Taktperiode
- Ausblick: warum Pipelining nötig wird

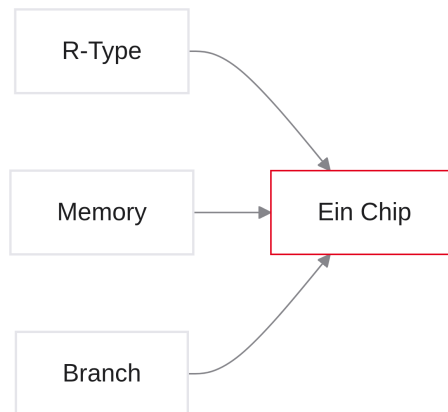


### Notizen

Finale Theorie: Steuerwerk und Datenpfad zusammen. Das vereinfachte Gesamtbild folgt nach der Ressourcen-/MUX-Zusammenfassung; danach Timing und Pipelining-Ausblick.

## Der Ressourcenkonflikt (Zusammenfassung)

- Drei Teilpfade: R-Type, Load/Store, Branch
- Problem: keine eigene ALU/Register File pro Befehlstyp
- Transistoren kosten Fläche, Energie und Geld
- Lösung: Pfade überlagern — gemeinsame Nutzung (Reuse)
- Dieselbe ALU: mal Addition, mal Adresse, mal Vergleich

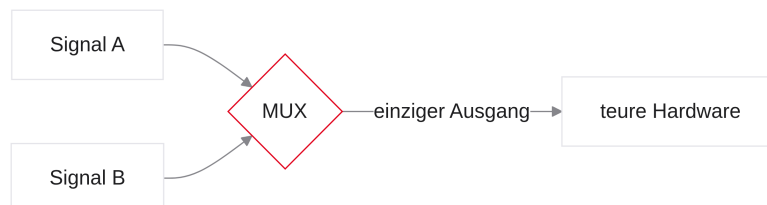


### Notizen

Drei ALUs wären absurd teuer. Eine ALU dient nacheinander ganz unterschiedlichen Befehlen — das ist die Genialität der Architektur.

## Multiplexer: Die Weichensteller

- Wie vermeiden wir Chaos beim Zusammenlegen der Kabel?
- Antwort: unsere drei Multiplexer
- Verkehrspolizisten des Datenpfads
- Ohne sie: Kurzschlüsse an teuren Eingängen (ALU, PC, Register)



### Notizen

Zwei Quellen, ein Ziel: Kabel aufschneiden, MUX dazwischen, ein sauberes Ausgangskabel.

## Der komplette Datenpfad (Patterson & Hennessy)

- Single-Cycle Datapath nach **Patterson und Hennessy** (RISC-V Edition; Signalnamen dieses Kurses)
- Harris & Harris (Kursliteratur) nutzen teils andere Namen (`ResultSrc` statt `MemtoReg`, `ImmSrc`, ...) — Konzept identisch
- Vereint IF, ID, EX, MEM, WB; drei MUX-Weichen hervorgehoben
- Rote gestrichelte Linien: Steuersignale der Control Unit

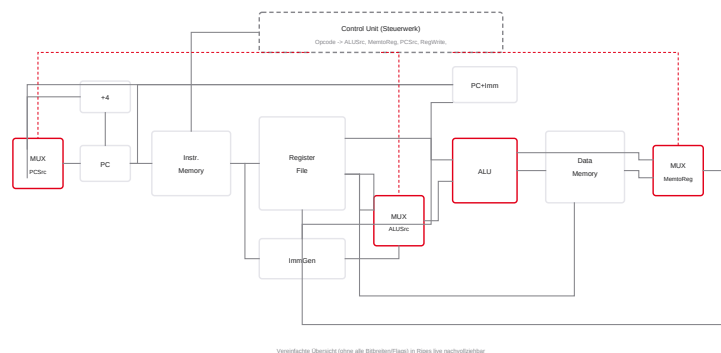


Abbildung 1: Single-Cycle-Datenpfad (vereinfacht)

### Notizen

Meisterdiagramm der Rechnerarchitektur — hier didaktisch verdichtet. Signalnamen folgen Patterson/Hennessy; in Ripes heißen einige Signale anders (Mapping-Folie).

## P&H-Diagramm und Ripes: Mapping

Das Lehrbuch-Diagramm und der Ripes-Single-Cycle-Prozessor sind **nicht baugleich**. Merken Sie sich die Entsprechungen:

| Patterson/Hennessy (Folien)                            | Ripes (Single-Cycle)                                                                                                                           |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| ALU-Zero + AND mit Branch $\rightarrow$ PCSrc          | Dedizierte <b>Branch-Vergleichseinheit</b> (alle Branch-Typen)                                                                                 |
| Separater Branch-Addierer<br>2-Wege-MUX MemtoReg       | Branch-Ziel oft über <b>Haupt-ALU</b> (PC als Op1)<br>Oft <b>3-Wege-Write-Back-MUX</b> (Reg wr<br>src: ALU / Mem / PC+4)                       |
| Signalnamen MemtoReg, MemRead, ...<br>Farben: beliebig | teils andere Bezeichner (Tooltips lesen!)<br>1-Bit-Signale: <b>grün = 1</b> , <b>grau = 0</b> (kein „rot = 0“, kein „ausgegraut = $\bar{X}$ “) |

Don't Care ( $\bar{X}$ ) ist ein **Entwurfskonzept** der Wahrheitstabelle — der Simulator zeigt immer einen konkreten Bitwert.

### Notizen

Labor: Paper-&-Pencil nach P&H, Verifikation in Ripes mit Mapping. Nicht erwarten, dass Zero+AND und Branch-Addierer 1:1 sichtbar sind.

## Weiche 1: ALUSrc

- Vor der Haupt-ALU
- Entscheidung: womit rechnen?
- 0: zweites Register (RD2)
- 1: Immediate aus ImmGen

```
add  t0, t1, t2  -> ALUSrc = 0
addi t0, t1, 5   -> ALUSrc = 1
```

### Notizen

R-Type: Registerkabel. I-Type/lw: ImmGen-Pfad.

### Weiche 2: MemtoReg

- Rechts vor dem Write-Back
- Entscheidung: was nach `rd`?
- 0: ALU-Ergebnis
- 1: RAM-Daten
- Bei `sw/beq`: Stellung oft Don't Care (weil `RegWrite = 0`)

```
add t0, t1, t2  -> MemtoReg = 0
lw  t0, 0(s1)   -> MemtoReg = 1
```

### Notizen

Ohne `RegWrite` prallt Müll am Register File ab — Don't Care ist ungefährlich.

### Weiche 3: PCSrc

- Oben links vor dem PC
- 0:  $PC + 4$  (Fall-Through)
- 1: Branch Target
- Einzige Weiche, die auch vom ALU-Zero abhängt (AND mit Branch)

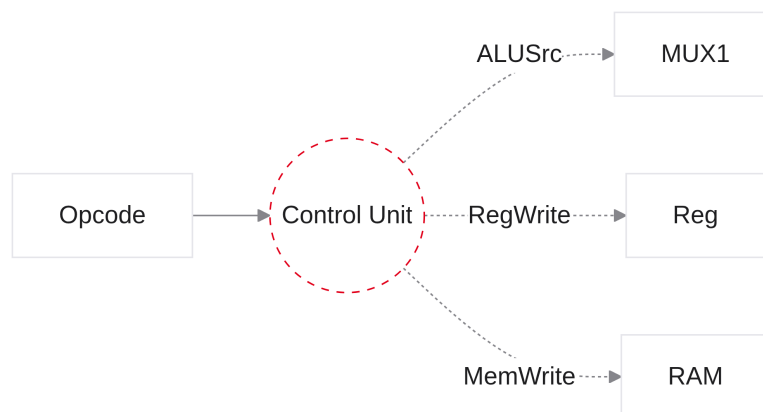
```
add ...          -> PCSrc = 0
beq ... (genommen) -> PCSrc = 1
```

### Notizen

Navigator des Prozessors: linear weiter oder Sprung — Control plus Live-Ergebnis der ALU.

## Die Steuersignale (Control)

- Typisch: RegWrite, MemRead, MemWrite, ALUSrc, ALUOp, ...
- Quelle: Control Unit (Steuerwerk)
- Liest den Opcode und erzeugt die Weichen-/Speicher-Signale
- Detailbau der Control Unit: Einheit 6

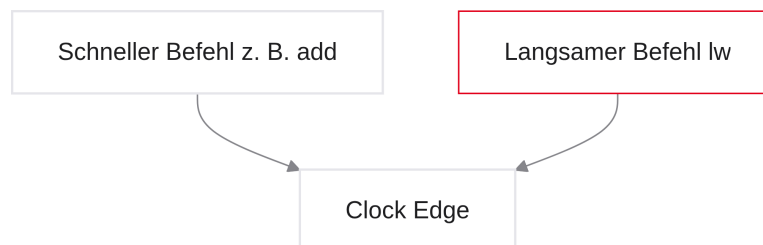


### Notizen

Dirigent von oben: Opcode analysieren, Multiplexer und Schreibfreigaben in Nanosekunden setzen.

## Das Timing-Problem (Critical Path)

- Single-Cycle: alles in einem Takt
- Nächste Flanke erst, wenn das letzte Signal stabil ist
- Taktperiode  $\geq$  kritischer Pfad (längste Verzögerungskette)
- Zu schneller Takt: nächster Fetch vor Write-Back — Absturz



### Notizen

Die Taktfrequenz muss zum langsamsten Befehl passen — Critical Path diktiert die MHz.

### Welcher Befehl ist der langsamste?

- `add`: Fetch → Decode → ALU → Write-Back (RAM idle)
- `beq`: Fetch → Decode → ALU/Branch-Addierer → PC
- Verlierer: `lw` — alle Stationen inkl. Data-Memory
- Gesamte CPU muss so langsam takten, dass `lw` in einen Zyklus passt



### Notizen

Zwei Speicherzugriffe plus langes Write-Back-Kabel: `lw` bestimmt den kritischen Pfad.

### Das Ende des Single-Cycle

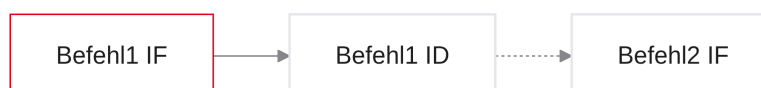
- Langer Takt für `lw` verschwendet Zeit bei schnellen Befehlen
- `add` ist früh fertig — ALU wartet bis zum Tick
- Ineffiziente Hardware-Nutzung
- Für hohe GHz: Architektur umbauen

### Notizen

Todesstoß des Single-Cycle in der Praxis: teures Silizium steht oft still.

### Ausblick: Pipelining (Fließband)

- Langen Zyklus in fünf kurze Takte zerlegen
- Fließband: während Befehl 1 in ID ist, startet Befehl 2 in IF
- Alle Blöcke arbeiten gleichzeitig an verschiedenen Befehlen
- Vertiefung: Einheit 7



### Notizen

Heute bleibt das übersichtliche Single-Cycle-Modell. Pipelining kommt in Einheit 7.

## Praxis: Ripes Onboarding

- Von abstrakten Diagrammen zur Simulation
- Tool: Ripes — Code-Editor plus Live-Hardware
- Stromsignale auf Leitungen ablesen
- Step-/Clock-Debugging auf den Gattern

### Notizen

Venus bleibt für Software. Ripes macht das Lehrbuchdiagramm interaktiv.

## Venus vs. Ripes

- Venus: Software-Fokus, Algorithmen, Register/RAM-Ergebnisse
- Ripes: Hardware-Fokus, Bits auf Leitungen, MUX und ALU live
- Code in Ripes etwas unbequemer — visuelles Feedback unschlagbar

```
Venus = Blick des Programmierers  
Ripes = Blick des Chip-Designers
```

### Notizen

Tool nach Fragestellung wählen: Algorithmus → Venus; Datenpfad → Ripes.

## Die UI: Prozessorwahl

- Zuerst den richtigen Chip einbauen
- Ripes bietet Single-Cycle und Pipeline-Varianten
- Heute zwingend: Single-Cycle Processor
- 5-Stage Pipeline wäre für heute irreführend

### Notizen

Chip-Icon oben links: Single-Cycle wählen. Das ist das Ripes-Modell der installierten Version, nicht bitgleich das Folien-Subset. Signalnamen von Patterson/Hennessy, Harris/Harris und Ripes nebeneinander lesen.

## Die UI: Editor-Tab

- Assembler wie in Venus (RISC-V)
- Kleine Tests: `add`, `lw`, `beq`
- Ripes assembliert oft live (kein separater Compile-Klick)

```
.text
li t0, 10
li t1, 15
add t2, t0, t1
```

### Notizen

`.text`-Direktiven und Mnemonics wie gewohnt. Danach Processor-Tab öffnen.

## Die UI: Datenpfad-Reiter

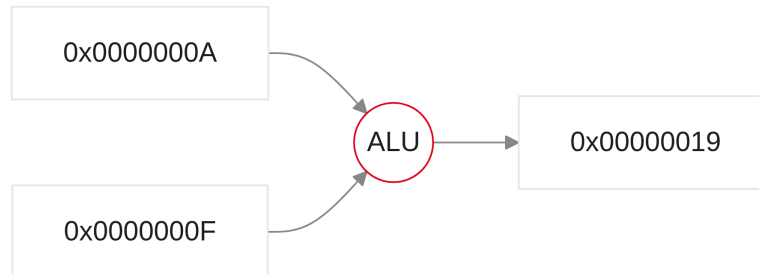
- Herzstück: Lehrbuchdiagramm (Patterson & Hennessy) live
- Dicke Linien: 32-Bit-Daten
- Tracing: Clock/Step — einen Takt nach dem anderen
- Run vermeiden, wenn Sie Signale verstehen wollen

### Notizen

PC links, ALU Mitte, Data Memory rechts — Clock-Tick treibt exakt einen Zyklus.

### Signale live ablesen

- Hex-Werte auf den Leitungen sichtbar
- Datenleitungen vs. 1-Bit-Steuersignale
- Beispiel:  $t0 = 10, t1 = 15 \rightarrow$  ALU-Ausgang  $0x19$  (25)



### Notizen

Maus über Werte: oft Dezimal-Tooltip. Theorie wird messbar.

### Multiplexer in Ripes untersuchen

- Bei `lw`: MUX vor der ALU (`ALUSrc`) suchen
- Steuersignal prüfen: steht es auf 1?
- Beobachtung: ImmGen-Pfad wird zur ALU durchgeschaltet

### Notizen

Visuelles Feedback: Control Unit zieht am Faden, Weiche klappt — Theorie greifbar.

### Register und RAM inspizieren

- Seitenleisten: Registers und Memory (wie Venus)
- Bei `sw`: MemWrite im Diagramm auf 1, Byte im Memory-Tab aktualisiert

```
Kette fuer sw t0, 0(s1):
```

1. Code tippen
2. Clock
3. MemWrite = 1 im Datenpfad
4. Memory-Tab: Wert liegt im RAM

## Notizen

Diagramm und Tabellen zusammen lesen — Store wird physisch sichtbar.

## Folie

77/79

### Labor-Aufgabe 1: R-Type-Trace

- Code: `li t0, 50 / li t1, 100 / add a0, t0, t1`
- Bis zum `add` steppen
- Fragen: Hex auf RD1? Wert von `ALUSrc`? Wert von `RegWrite`?

## Notizen

Reine Analyse, kein schweres Programmieren. Stimmen die Leitungen mit der Theorie?

## Folie

78/79

### Labor-Aufgabe 2: Der Load-Trace

- `.data` mit `.word 99, 88`
- Base in `s1`, dann `lw a0, 4(s1)`
- `ImmGen`: wird 4 zu 32-Bit?
- Welche Adresse berechnet die ALU?
- Stellung von `MemtoReg` ganz rechts?

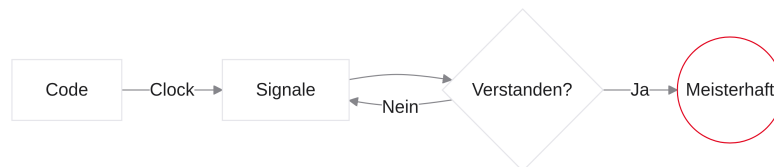
```
.data
werte: .word 99, 88
.text
    la s1, werte
    lw a0, 4(s1)
```

## Notizen

Kritischer Pfad live: `ImmGen`, Adress-ALU, RAM, `MemtoReg` — langsamster Befehl.

## Q&A und Hands-on

- Ripes starten (Browser oder lokal)
- Single-Cycle Processor einstellen
- Leitungen explorieren — die CPU ist kein Mysterium mehr
- Viel Erfolg beim Hardware-Hacking



## Notizen

Damit endet Einheit 5: Assembler trifft Silizium. Im Labor Single-Cycle checken und den Datenpfad auseinandernehmen.